# Diseño Arquitectónico y Evaluación de un Sistema Basado en MVC/MVVM con Stack Empresarial

Pontificia Universidad JAVERIANA Colombia

Arquitectura de Software

Período Académico 2026-1

**Integrantes de Grupo:**
Daniel Felipe Bohorquez Casas
Juan Esteban Diaz Toledo
Juan Eduardo Diaz Rojas

Bogotá - Colombia

13 de abril de 2026

**Tabla de Contenido:**


1. Introducción ..... 2
2. Descripción del Estilo Arquitectónico ..... 2
   2.1 Definición ..... 3
   2.2 Clasificación ..... 3
   2.3 Características principales ..... 4
   MVC (Model–View–Controller) ..... 4
   MVP (Model–View–Presenter) ..... 4
   MVVM (Model–View–ViewModel) ..... 5
   2.4 Historia y evolución ..... 5
   2.5 Ventajas y desventajas ..... 6
   Ventajas ..... 6
   Desventajas ..... 6
   2.6 Problemas comunes ..... 7
   2.7 Patrones asociados ..... 8
   Patrones de persistencia y gestión de estado ..... 8
   Patrones de comunicación e integración ..... 9
   Patrones de eventos y comportamiento ..... 9
   Estrategias de renderizado y UI ..... 9
   Patrones de diseño de código (GoF) ..... 10
   2.8 Casos de uso ..... 10
   Cuándo usarlo ..... 10
   Cuándo NO usarlo ..... 10
   2.9 Casos reales en la industria ..... 11
   Caso 1: Plataforma Upwork (Industria: Marketplace digital / Freelance) ..... 11
   Caso 2: Forbes (Industria: Medios digitales / Publicación) ..... 11
   Caso 3: NASA (Industria: Gobierno / Ciencia y tecnología) ..... 11
   Caso 4: Plataformas de e-commerce (Industria: Retail digital) ..... 12
   Caso 5: Ecosistema Angular (Industria: desarrollo web / open source) ..... 12
3. Descripción del Stack Tecnológico ..... 12

3.1 Frontend: Angular ..... 12
3.2 Backend: Spring Boot ..... 13
3.3 Base de datos: Cassandra ..... 15
3.4 Protocolo de comunicación: WebSocket ..... 16
3.5 Relación entre tecnologías ..... 18
3.6 Popularidad del stack ..... 19
4. Análisis Arquitectónico ..... 20
4.1 Matriz: Atributos de calidad vs estilo ..... 21
4.2 Matriz: Principios vs estilo ..... 23
4.3 Matriz: Tácticas vs estilo y stack ..... 25
4.4 Matriz: Mercado laboral ..... 27
5. Diseño del Sistema (C4 Model + HLD) ..... 29
5.1 Descripción del sistema ..... 29
5.2 Diagrama de Alto Nivel (HLD) ..... 30
5.3 Diagrama de Contexto (Nivel 1) ..... 30
5.4 Diagrama de Contenedores (Nivel 2) ..... 31
5.5 Diagrama dinámico ..... 31
5.6 Diagrama de despliegue ..... 32
6. Implementación ..... 32
6.1 Descripción del proyecto ..... 32
6.2 Arquitectura implementada ..... 33
6.3 Flujo funcional ..... 34
6.4 Manejo de errores y validaciones ..... 35
6.5 Integración entre componentes ..... 36
7. Repositorio y DevOps ..... 38
8. Lecciones Aprendidas ..... 39
9. Conclusiones ..... 40
10. Referencias ..... 40

# 1. Introducción

El presente documento se desarrolla en el marco del taller “Presentación 1: Exploración de Estilos Arquitectónicos y Stack Tecnológico”, correspondiente a la asignatura de Arquitectura

de Software. Esta actividad tiene como propósito fortalecer las competencias en el análisis, diseño e implementación de soluciones de software a partir del estudio de estilos arquitectónicos reales y el uso de stacks tecnológicos representativos de la industria. En este contexto, el trabajo se enfoca en la exploración de los patrones de presentación (MVC, MVP y MVVM), así como en su relación con tecnologías modernas como Angular en el frontend, Spring Boot en el backend y Cassandra como motor de persistencia, integrados mediante un protocolo de comunicación tipo REST.

El objetivo de este documento es analizar, justificar y aplicar de manera coherente un estilo arquitectónico junto con un stack tecnológico específico, utilizando criterios técnicos y buenas prácticas de la ingeniería de software. Para ello, se aborda la investigación del estilo y las tecnologías seleccionadas, el análisis arquitectónico mediante matrices de evaluación, el modelado de la solución utilizando el modelo C4 y, finalmente, la implementación de un sistema funcional que evidencie la correcta integración de todos los componentes.

Como parte del desarrollo del taller, se propone la construcción de un sistema de software funcional que permita gestionar información a través de una arquitectura cliente-servidor, donde el frontend desarrollado en Angular interactúa con un backend implementado en Spring Boot, el cual a su vez gestiona la persistencia de datos en Cassandra. Este sistema contemplará al menos tres entidades de negocio interrelacionadas y permitirá ejecutar un flujo completo de extremo a extremo, incluyendo operaciones de creación, consulta y validación de datos, garantizando consistencia con el estilo arquitectónico seleccionado y aplicando principios de diseño adecuados.

En conjunto, este documento integra los componentes de investigación, análisis, diseño e implementación requeridos, con el fin de evidenciar una toma de decisiones arquitectónicas fundamentada, alineada tanto con las características del estilo seleccionado como con las capacidades y limitaciones del stack tecnológico definido. Asimismo, se busca demostrar una visión integral del proceso de construcción de software, desde la conceptualización hasta la materialización de una solución funcional y técnicamente sólida.

# 2. Descripción del Estilo Arquitectónico

## 2.1 Definición

El patrón Model–View–Controller (MVC) es un patrón de arquitectura de software que separa una aplicación en tres componentes principales: el modelo, que gestiona los datos y la lógica de negocio; la vista, que representa la interfaz de usuario; y el controlador, que actúa como intermediario entre ambos, gestionando las entradas del usuario y actualizando el estado del sistema. Esta separación permite mejorar la mantenibilidad, escalabilidad y reutilización del código, siendo ampliamente utilizado tanto en aplicaciones web como en frameworks backend modernos (Gamma et al., 1994; Buschmann et al., 1996).

El patrón Model–View–Presenter (MVP) es una variante de MVC diseñada para mejorar la separación de responsabilidades, especialmente en interfaces de usuario complejas. En este patrón, el presenter asume toda la lógica de interacción, comunicándose directamente con la vista mediante interfaces, lo que reduce el acoplamiento entre la vista y el modelo. A diferencia de MVC, la vista en MVP es más pasiva, delegando completamente el manejo de eventos y lógica al presenter, lo que facilita las pruebas unitarias y el desarrollo modular (Fowler, 2004).

El patrón Model–View–ViewModel (MVVM) es una evolución de los patrones de presentación orientada a aplicaciones modernas, especialmente en frameworks frontend. En este patrón, el ViewModel actúa como intermediario entre el modelo y la vista, proporcionando un enlace automático mediante mecanismos como el data binding. Esto permite que los cambios en el modelo se reflejen automáticamente en la vista sin necesidad de intervención manual. MVVM es ampliamente utilizado en tecnologías como Angular, donde la reactividad y la sincronización de datos son fundamentales (Microsoft, 2012; Freeman & Robson, 2014).

Qué NO son estos patrones:

- No son arquitecturas completas del sistema (como microservicios o monolitos), sino patrones de presentación enfocados en la capa de interfaz.
- No definen la infraestructura, despliegue o comunicación entre sistemas (ej: REST, gRPC).
- No eliminan la necesidad de otras capas arquitectónicas (como backend, persistencia o integración).
- No son intercambiables directamente, ya que cada uno responde a diferentes necesidades de desacoplamiento y complejidad en la interfaz.

## 2.2 Clasificación

Los patrones MVC, MVP y MVVM pertenecen a la categoría de patrones de presentación (Frontend/UI), los cuales se enfocan en organizar la interacción entre la interfaz de usuario y la lógica de la aplicación. Su objetivo principal es separar responsabilidades dentro de la capa de presentación, facilitando el mantenimiento, la escalabilidad y la evolución del sistema sin afectar otras capas como la lógica de negocio o la persistencia (Buschmann et al., 1996).

Dentro de esta clasificación, estos patrones se consideran soluciones estructurales para el diseño de interfaces, ya que definen cómo se distribuyen las responsabilidades entre los componentes que interactúan directamente con el usuario. En el contexto de aplicaciones modernas, especialmente aquellas basadas en arquitecturas cliente-servidor, estos patrones son fundamentales para garantizar una correcta organización del frontend y su integración con el backend. Además, permiten aplicar principios de diseño como separación de responsabilidades y bajo acoplamiento, los cuales son esenciales en el desarrollo de software escalable y mantenible (Fowler, 2004).

Adicionalmente, estos patrones de presentación pueden entenderse como una evolución progresiva en la forma de desacoplar la interfaz de usuario de la lógica de negocio, donde cada uno responde a diferentes niveles de complejidad y necesidades tecnológicas. Mientras MVC surgió en aplicaciones tradicionales con menor dinamismo en la interfaz, MVP introdujo un mayor control sobre la interacción para mejorar la testabilidad, y MVVM se consolidó con la aparición de frameworks modernos que requieren alta reactividad y sincronización automática de datos.

## 2.3 Características principales

Los patrones de presentación MVC, MVP y MVVM comparten el objetivo de separar responsabilidades dentro de la capa de interfaz, pero difieren en la forma en que gestionan la interacción entre sus componentes. A continuación, se describen sus características principales:

**MVC (Model–View–Controller)**

El patrón MVC se caracteriza por una separación clara entre datos, interfaz y control de flujo, donde el controlador actúa como intermediario entre el usuario y el sistema.

- Separación en tres componentes: modelo, vista y controlador.
- El controlador gestiona la entrada del usuario y actualiza el modelo.
- La vista se actualiza a partir de los cambios en el modelo.
- Permite múltiples vistas para un mismo modelo.
- Acoplamiento moderado entre vista y controlador.
- Adecuado para aplicaciones web tradicionales y frameworks backend.

**MVP (Model–View–Presenter)**

El patrón MVP introduce un mayor desacoplamiento al delegar toda la lógica de interacción en el presenter, haciendo que la vista sea completamente pasiva.

- El presenter actúa como intermediario central entre modelo y vista.
- La vista no contiene lógica, solo renderiza información.
- Comunicación basada en interfaces (bajo acoplamiento).
- Facilita pruebas unitarias (testabilidad alta).
- Mayor control sobre el flujo de la aplicación.
- Incrementa la complejidad estructural respecto a MVC.

**MVVM (Model–View–ViewModel)**

El patrón MVVM está orientado a entornos modernos y se basa en la sincronización automática entre vista y modelo mediante mecanismos reactivos.

- Uso de **data binding** para sincronización automática.

- El ViewModel expone datos y comportamiento a la vista.
- Separación clara entre lógica de presentación y vista.
- Alta reactividad (ideal para aplicaciones SPA).
- Reduce la necesidad de código imperativo en la vista.
- Muy utilizado en frameworks modernos como Angular.

## 2.4 Historia y evolución

Los patrones de presentación han evolucionado como respuesta a la creciente complejidad de las interfaces de usuario y la necesidad de mejorar la mantenibilidad y escalabilidad del software. El patrón **MVC** fue introducido originalmente en la década de 1970 en el entorno de desarrollo Smalltalk en Xerox PARC, como una solución para separar la lógica de negocio de la interfaz gráfica en aplicaciones interactivas. Posteriormente, fue formalizado y popularizado en la literatura de ingeniería de software, convirtiéndose en un estándar para el diseño de aplicaciones con interfaz de usuario (Krasner & Pope, 1988; Gamma et al., 1994).

Con el paso del tiempo, las limitaciones de MVC, especialmente en términos de acoplamiento entre la vista y el controlador, dieron lugar a la aparición de variantes más especializadas. En este contexto, surge el patrón **MVP**, ampliamente documentado por Martin Fowler, como una alternativa que mejora la separación de responsabilidades al introducir el presenter como intermediario principal. Este enfoque fue adoptado principalmente en aplicaciones de escritorio y entornos donde la testabilidad era un requisito crítico, permitiendo aislar la lógica de presentación de la interfaz gráfica (Fowler, 2004).

Posteriormente, con la evolución hacia aplicaciones web dinámicas y frameworks modernos, se desarrolló el patrón **MVVM**, impulsado principalmente por Microsoft en el contexto de tecnologías como WPF y Silverlight. MVVM introduce el concepto de *data binding* bidireccional, permitiendo una sincronización automática entre la vista y el modelo, lo que reduce significativamente el código de control manual y facilita la creación de interfaces altamente reactivas. Este patrón se ha consolidado en el desarrollo frontend moderno, siendo ampliamente adoptado en frameworks como Angular debido a su capacidad para manejar estados complejos de manera eficiente (Microsoft, 2012; Freeman & Robson, 2014).

- **Década de 1970–1980:** Aparición de MVC en Smalltalk (Xerox PARC).
- **Década de 1990:** Formalización de MVC en patrones de diseño (GoF).
- **Años 2000:** Surgimiento de MVP como mejora en desacoplamiento y testabilidad.
- **Años 2010 en adelante:** Consolidación de MVVM con frameworks modernos y aplicaciones SPA.

Esta evolución refleja una tendencia hacia arquitecturas cada vez más desacopladas, reactivas y orientadas a la experiencia del usuario, donde los patrones de presentación se adaptan a las necesidades tecnológicas de cada época. Actualmente, MVVM representa el enfoque predominante en el desarrollo frontend moderno, mientras que MVC continúa siendo fundamental en el backend y en arquitecturas tradicionales.

## 2.5 Ventajas y desventajas

Los patrones de presentación MVC, MVP y MVVM aportan una estructura clara para la organización de la capa de interfaz, permitiendo separar responsabilidades y mejorar la calidad del software. Sin embargo, su adopción también implica ciertos compromisos en términos de complejidad y sobrecarga estructural. A continuación, se presentan sus principales ventajas y desventajas de manera consolidada:

**Ventajas**

- **Separación de responsabilidades (SoC):** Estos patrones permiten dividir claramente la lógica de negocio, la interfaz de usuario y el control de flujo, lo que facilita la mantenibilidad del sistema. Esta separación reduce el impacto de cambios, ya que modificaciones en la interfaz no afectan directamente la lógica de negocio.
- **Mayor mantenibilidad y escalabilidad:** Al estructurar el código en componentes bien definidos, es posible escalar el sistema de manera ordenada. Esto es especialmente relevante en aplicaciones empresariales donde múltiples desarrolladores trabajan simultáneamente.
- **Reutilización de componentes:** El modelo puede ser reutilizado por múltiples vistas (en MVC) o el ViewModel puede ser compartido entre diferentes componentes (en MVVM), lo que reduce la duplicación de código.
- **Facilidad para pruebas (testabilidad):** En patrones como MVP y MVVM, la lógica de presentación está desacoplada de la vista, lo que permite realizar pruebas unitarias sin necesidad de depender de la interfaz gráfica.
- **Mejor organización del código:** Facilitan una estructura modular que promueve buenas prácticas de diseño como bajo acoplamiento y alta cohesión, alineándose con principios como SOLID.
- **Soporte para aplicaciones modernas:** MVVM, en particular, permite implementar interfaces reactivas mediante mecanismos como data binding, lo que mejora la experiencia de usuario en aplicaciones SPA.

**Desventajas**

- **Aumento de la complejidad estructural:** La separación en múltiples capas o componentes puede hacer que el sistema sea más complejo de entender, especialmente para desarrolladores con poca experiencia en estos patrones.
- **Sobrecarga de código (boilerplate):** La implementación de estos patrones suele requerir la creación de múltiples clases (controllers, presenters, viewmodels, etc.), lo que incrementa la cantidad de código inicial necesario.
- **Curva de aprendizaje:** Comprender correctamente las responsabilidades de cada componente puede ser difícil al inicio, especialmente al diferenciar entre patrones similares como MVC, MVP y MVVM.

- **Posible sobreingeniería:** En aplicaciones pequeñas o simples, aplicar estos patrones puede resultar innecesario, generando una arquitectura más compleja de lo requerido.
- **Gestión de estados complejos:** En aplicaciones grandes, especialmente con MVVM, el manejo del estado puede volverse difícil si no se utilizan herramientas adicionales o buenas prácticas.
- **Acoplamiento indirecto (dependencias implícitas):** Aunque buscan desacoplar componentes, en algunos casos (como MVC tradicional) puede existir dependencia implícita entre la vista y el modelo, lo que afecta la flexibilidad.

## 2.6 Problemas comunes

A pesar de sus beneficios, la implementación de patrones como MVC, MVP y MVVM suele presentar una serie de problemas recurrentes, especialmente cuando no se aplican correctamente o se adaptan de forma inadecuada al contexto del sistema. Estos problemas pueden afectar la calidad, mantenibilidad y evolución del software, y en muchos casos están directamente relacionados con antipatrones ampliamente documentados en la industria.

- **Mala asignación de responsabilidades:** Uno de los errores más comunes es ubicar lógica de negocio en la vista o en componentes incorrectos (por ejemplo, lógica en controladores o componentes frontend), lo que rompe el principio de separación de responsabilidades. Este problema está estrechamente relacionado con el antipatrón *Anemic Domain Model*, donde las entidades carecen de lógica, y con malas prácticas que derivan en arquitecturas poco cohesionadas.

- **Controladores o componentes "sobrecargados":** En implementaciones deficientes de MVC o MVVM, es frecuente que los controladores o componentes acumulen demasiadas responsabilidades, convirtiéndose en puntos críticos difíciles de escalar o modificar. Este problema se asocia directamente con el antipatrón *God Object*, donde una sola clase concentra gran parte de la lógica del sistema, afectando la mantenibilidad y escalabilidad.

- **Exceso de acoplamiento entre capas (Tight Coupling):** Aunque estos patrones buscan desacoplar componentes, una mala implementación puede generar dependencias fuertes entre la vista, el modelo y el controlador o ViewModel. Este problema corresponde al antipatrón *Tight Coupling*, el cual dificulta la reutilización de componentes, limita la flexibilidad del sistema y complica su evolución.

- **Duplicación de lógica y falta de cohesión:** En algunos casos, la lógica de validación o transformación de datos se replica entre diferentes capas (frontend y backend), lo que incrementa el riesgo de inconsistencias y errores. Este problema puede derivar en un sistema desorganizado cercano al antipatrón *Spaghetti Architecture* o incluso a un *Big Ball of Mud*, donde no existe una estructura clara ni separación adecuada de responsabilidades.

- **Dificultad en el manejo del estado (especialmente en MVVM):** En aplicaciones complejas, el estado puede dispersarse entre múltiples componentes o ViewModels, dificultando su control, trazabilidad y depuración. Si no se gestiona adecuadamente, este problema puede escalar a escenarios de sobreingeniería (*Overengineering*), donde se introducen soluciones innecesariamente complejas para manejar el estado.

- **Falta de claridad en el flujo de datos:** Especialmente en implementaciones basadas en MVVM con data binding, el flujo de información puede volverse implícito y poco transparente. Esto dificulta la comprensión del sistema y su depuración, afectando la productividad del equipo de desarrollo.

- **Implementación incorrecta del patrón:** Es común encontrar sistemas que afirman utilizar MVC o MVVM, pero que en la práctica mezclan responsabilidades o no respetan la estructura del patrón. Esto conduce a arquitecturas inconsistentes y difíciles de mantener, muchas veces asociadas al antipatrón *Golden Hammer*, donde se intenta aplicar un mismo enfoque sin considerar el contexto real del problema.

- **Sobreuso del patrón (Overengineering):** Aplicar estos patrones en sistemas simples o con baja complejidad puede generar una arquitectura innecesariamente compleja, aumentando el tiempo de desarrollo y dificultando la comprensión del sistema sin aportar beneficios reales.

## 2.7 Patrones asociados

Los patrones de presentación como MVC, MVP y MVVM no funcionan de manera aislada, sino que se apoyan en otros patrones arquitectónicos y de diseño que permiten resolver problemas específicos relacionados con la persistencia, la comunicación, la interacción entre componentes y la construcción de interfaces modernas. A continuación, se presentan los patrones más relevantes asociados a este estilo, seleccionados en función de su aplicabilidad al stack tecnológico definido.

**Patrones de persistencia y gestión de estado**

- **Repository:** Este patrón encapsula la lógica de acceso a datos, permitiendo desacoplar la capa de negocio del mecanismo de persistencia. En el contexto de MVC con Spring Boot, el Repository es fundamental para mantener una arquitectura limpia, donde los controladores no interactúan directamente con la base de datos. Se relaciona directamente con el modelo (Model), ya que abstrae la persistencia de entidades.
- **DAO (Data Access Object):** Similar al Repository, permite aislar la lógica de acceso a datos, especialmente útil en sistemas donde se requiere un control más explícito sobre las operaciones de base de datos. Es clave para mantener la separación de responsabilidades dentro del patrón MVC.

- **CQRS (Command Query Responsibility Segregation):** Este patrón separa las operaciones de lectura y escritura, lo cual es especialmente relevante cuando se utilizan bases de datos NoSQL como Cassandra. Se relaciona con MVVM y arquitecturas modernas al optimizar el manejo del estado y las consultas en aplicaciones escalables.

**Patrones de comunicación e integración**

- **BFF (Backend for Frontend):** Este patrón propone crear un backend específico para el frontend, adaptado a sus necesidades. En el contexto de Angular, permite optimizar las respuestas del backend y simplificar la lógica en el frontend. Se relaciona con MVVM al facilitar la comunicación eficiente entre la vista y el modelo.
- **Facade:** Proporciona una interfaz simplificada para un conjunto de interfaces más complejas. En el backend, puede utilizarse para simplificar la interacción entre controladores y servicios. Ayuda a mantener bajo acoplamiento y simplificar la lógica en MVC.
- **API REST (estilo arquitectónico complementario):** Aunque no es un patrón GoF, es fundamental como mecanismo de comunicación entre frontend y backend. Permite implementar el modelo cliente-servidor que conecta Angular con Spring Boot.

**Patrones de eventos y comportamiento**

- **Observer:** Permite que múltiples componentes reaccionen a cambios en el estado de un objeto. Este patrón es clave en MVVM, especialmente en Angular con el uso de programación reactiva (RxJS). Es la base conceptual del data binding y la reactividad.
- **Publish–Subscribe (Pub/Sub):** Similar a Observer, pero desacoplado mediante eventos. Puede utilizarse para manejar eventos en el frontend o incluso en arquitecturas más distribuidas. Mejora la escalabilidad y desacoplamiento del sistema.

**Estrategias de renderizado y UI**

- **CSR (Client-Side Rendering):** En este enfoque, la renderización ocurre en el navegador, lo cual es característico de Angular. Se alinea con MVVM, ya que la lógica de presentación reside en el cliente.
- **Data Binding:** Aunque no es un patrón formal GoF, es un mecanismo clave en MVVM que permite sincronizar automáticamente la vista y el modelo. Es fundamental para reducir la complejidad del código en la interfaz.

**Patrones de diseño de código (GoF)**

- **Strategy:** Permite definir diferentes comportamientos intercambiables, útil para manejar lógica variable en controladores o servicios.
- **Factory / Abstract Factory:** Facilita la creación de objetos sin acoplar el código a implementaciones concretas, muy usado en frameworks como Spring.

- **Decorator:** Permite añadir funcionalidades de forma dinámica, útil en servicios backend o middlewares.

## 2.8 Casos de uso

Los patrones de presentación como MVC, MVP y MVVM son ampliamente utilizados en el desarrollo de aplicaciones modernas, pero su aplicabilidad depende del contexto, complejidad y requerimientos del sistema. A continuación, se describen los principales escenarios donde su uso es adecuado y donde no resulta recomendable.

**Cuándo usarlo**

- **Aplicaciones con interfaz de usuario compleja:** Cuando se requiere manejar múltiples vistas, estados dinámicos y lógica de interacción, estos patrones permiten organizar el código de forma clara y mantenible.
- **Sistemas cliente-servidor (SPA + API):** En arquitecturas modernas donde el frontend (Angular) consume servicios backend (Spring Boot), el uso de MVVM y MVC facilita la separación de responsabilidades.
- **Proyectos con equipos grandes de desarrollo:** La separación en capas permite que diferentes equipos trabajen en frontend y backend sin interferencias, mejorando la productividad.
- **Aplicaciones que requieren alta mantenibilidad:** Sistemas que evolucionan constantemente se benefician de esta estructura, ya que los cambios en una capa no afectan directamente a las demás.
- **Aplicaciones con necesidad de pruebas unitarias:** Patrones como MVP y MVVM facilitan la testabilidad al desacoplar la lógica de la interfaz.
- **Sistemas escalables y de largo plazo:** Permiten una evolución progresiva del sistema sin necesidad de rediseños completos.

**Cuándo NO usarlo**

- **Aplicaciones pequeñas o prototipos simples:** La implementación de estos patrones puede introducir complejidad innecesaria y aumentar el tiempo de desarrollo.
- **Sistemas sin interfaz de usuario:** En aplicaciones puramente backend o servicios batch, estos patrones no aportan valor.
- **Proyectos con requerimientos muy simples:** Si la lógica de negocio y la interfaz son mínimas, la separación en múltiples capas puede ser innecesaria.
- **Equipos sin experiencia en patrones arquitectónicos:** Una mala implementación puede generar más problemas que beneficios, como acoplamiento indebido o código desorganizado.
- **Aplicaciones donde el rendimiento extremo es crítico y la abstracción es costosa:** En sistemas de muy baja latencia, la sobrecarga de capas puede ser un problema si no se gestiona correctamente.

## 2.9 Casos reales en la industria

Los patrones de presentación como MVC, MVP y MVVM han sido ampliamente adoptados en la industria del software, especialmente en aplicaciones web modernas y sistemas empresariales. Su uso permite estructurar aplicaciones complejas de manera organizada, facilitando la escalabilidad, mantenibilidad y evolución de los sistemas. A continuación, se presentan algunos casos reales relevantes donde estos patrones han sido aplicados en distintos contextos industriales.

**Caso 1: Plataforma Upwork (Industria: Marketplace digital / Freelance)**

La plataforma Upwork implementó una arquitectura frontend basada en Angular, apoyándose en principios similares a MVVM, lo que permitió estructurar la interfaz mediante componentes reactivos y desacoplados. Este enfoque facilitó la gestión de estados dinámicos y la interacción con APIs backend, mejorando la mantenibilidad del sistema. Como resultado, la plataforma logró optimizar el rendimiento y la experiencia de usuario, evidenciado en mejoras en tiempos de carga y navegación (VLink, 2024).

**Caso 2: Forbes (Industria: Medios digitales / Publicación)**

El portal digital de Forbes migró parte de su arquitectura hacia Angular, adoptando patrones de presentación como MVVM para gestionar contenido dinámico y altamente interactivo. Esta implementación permitió desacoplar la lógica de presentación de la interfaz, facilitando la personalización de contenido y la actualización en tiempo real. Como resultado, la empresa experimentó un aumento significativo en tráfico móvil y en ingresos publicitarios, evidenciando el impacto de una arquitectura bien estructurada en el negocio digital (VLink, 2024).

**Caso 3: NASA (Industria: Gobierno / Ciencia y tecnología)**

El sitio web de NASA utiliza Angular para construir interfaces altamente dinámicas e informativas, aplicando principios cercanos a MVVM para manejar grandes volúmenes de datos científicos y visualizaciones. Este enfoque permite separar claramente la lógica de datos del componente visual, facilitando la actualización constante de información en tiempo real. La adopción de este patrón ha permitido ofrecer una experiencia interactiva robusta para usuarios globales (VLink, 2024).

**Caso 4: Plataformas de e-commerce (Industria: Retail digital)**

En grandes plataformas de comercio electrónico (basadas en tecnologías como ASP.NET MVC o frameworks similares), el patrón MVC ha sido ampliamente utilizado para gestionar catálogos, pagos y flujos de compra. En un caso documentado, una plataforma de retail utilizó MVC para manejar rutas críticas como navegación de productos y procesamiento de pagos, mientras incorporaba componentes más dinámicos en el frontend. Este enfoque permitió

mejorar el rendimiento y aumentar la tasa de conversión, demostrando la relevancia de MVC en sistemas transaccionales (Dimension Zero, 2024).

**Caso 5: Ecosistema Angular (Industria: desarrollo web / open source)**

El proyecto Angular y sus aplicaciones reales (como repositorios de referencia tipo *RealWorld App*) demuestran el uso práctico de patrones como MVC y MVVM en aplicaciones completas con autenticación, CRUD y manejo de estados. Estos proyectos reflejan cómo en entornos reales se combinan estos patrones con arquitecturas cliente-servidor para construir aplicaciones escalables y mantenibles, siguiendo buenas prácticas ampliamente adoptadas en la industria del desarrollo web moderno.

# 3. Descripción del Stack Tecnológico

## 3.1 Frontend: Angular

**Definición**

Angular es un framework de desarrollo integral de nivel empresarial diseñado para la construcción de aplicaciones web de una sola página (SPA). Se distingue por ser una plataforma robusta que utiliza TypeScript de forma nativa y sigue fundamentalmente el patrón MVVM (Model-View-ViewModel) en su arquitectura interna para gestionar la sincronización entre los datos y la interfaz. No es solo una librería de interfaz, sino un ecosistema completo que incluye herramientas para el manejo de formularios, comunicaciones HTTP y enrutamiento.

**Características**

- **Programación Reactiva:** Integra de forma nativa la librería RxJS, permitiendo gestionar flujos de datos asíncronos de manera eficiente.
- **Data Binding:** Utiliza mecanismos de enlace de datos que permiten la sincronización automática entre el *ViewModel* y la *Vista*, reduciendo la necesidad de manipular el DOM manualmente.
- **Arquitectura basada en Componentes:** Organiza la interfaz en piezas independientes, reutilizables y desacopladas, lo que facilita el mantenimiento.
- **Inyección de Dependencias:** Posee un sistema sofisticado para proveer servicios a los componentes, promoviendo el bajo acoplamiento y facilitando las pruebas unitarias.
- **Renderizado del lado del Cliente (CSR):** Por defecto, la lógica de presentación y el renderizado ocurren en el navegador del usuario.

**Ventajas y Desventajas**

**Ventajas:**

- **Alta Reactividad:** Ideal para aplicaciones que requieren actualizaciones de datos en tiempo real sin recargar la página.
- **Escalabilidad:** Al ser un framework estructurado, permite que múltiples desarrolladores trabajen simultáneamente en proyectos grandes sin interferirse.
- **Mantenibilidad:** La clara separación de responsabilidades y el uso de TypeScript mejoran la calidad del código a largo plazo.

**Desventajas:**

- **Curva de Aprendizaje:** Es más compleja de aprender que otras alternativas debido a su gran cantidad de conceptos internos (decoradores, módulos, observables).
- **Sobreingeniería:** Para aplicaciones o prototipos muy simples, puede resultar innecesariamente pesado y generar una estructura excesiva.
- **Complejidad Estructural:** Requiere la creación de múltiples archivos y clases incluso para funcionalidades pequeñas.

**Casos de Uso**

- **Cuando usarlo:** En aplicaciones con interfaces de usuario complejas, sistemas de gestión empresarial, plataformas con estados dinámicos y proyectos donde se requiera testabilidad alta.
- **Cuando NO usarlo:** En sitios web estáticos simples, prototipos de corta duración o aplicaciones puramente backend donde no existe una interfaz interactiva.

**Ejemplos Reales**

1. **Upwork:** Utiliza Angular para estructurar su interfaz mediante componentes reactivos que gestionan estados dinámicos en su marketplace de freelancers.
2. **Forbes:** Adoptó Angular para manejar contenido dinámico y altamente interactivo, logrando mejorar su rendimiento en dispositivos móviles.
3. **NASA:** Emplea este framework para construir interfaces informativas que visualizan grandes volúmenes de datos científicos en tiempo real.
4. **Google:** Es el creador y principal usuario del framework para muchas de sus herramientas internas y públicas.

## 3.2 Backend: Spring Boot

**Definición**

Spring Boot es un framework de código abierto basado en Java (también compatible con Kotlin y Groovy) diseñado para simplificar el proceso de configuración y despliegue de aplicaciones de grado empresarial. No es un servidor de aplicaciones por sí mismo, sino un ecosistema que utiliza el concepto de "convención sobre configuración" para permitir que los desarrolladores creen aplicaciones "listas para producción" con una configuración manual mínima.

**Características**

- **Inyección de Dependencias:** Utiliza este patrón de forma nativa para gestionar el ciclo de vida de los objetos y fomentar el bajo acoplamiento.
- **Servidor Embebido:** Incluye servidores como Tomcat o Jetty por defecto, eliminando la necesidad de desplegar archivos WAR en servidores externos.
- **Ecosistema Robusto:** Integra herramientas de nivel empresarial para persistencia (Spring Data), seguridad (Spring Security) y manejo de transacciones.
- **Auto-configuración:** Escanea el proyecto y configura automáticamente los componentes basados en las dependencias añadidas al *classpath*.
- **Starters:** Proporciona descriptores de dependencias simplificados para agrupar tecnologías comunes (ej. spring-boot-starter-data-jpa).

**Ventajas y Desventajas**

**Ventajas:**

- **Estándar de la Industria:** Es considerado el "rey" en sectores críticos como la banca y los seguros debido a su estabilidad y soporte a largo plazo.
- **Productividad:** Reduce drásticamente el tiempo de desarrollo inicial al evitar el código repetitivo (*boilerplate*).
- **Seguridad:** Ofrece uno de los marcos de seguridad más completos y probados del mercado para proteger APIs y datos.

**Desventajas:**

- **Consumo de Recursos:** Al ejecutarse sobre la JVM (Java Virtual Machine), tiende a consumir más memoria RAM y tener tiempos de arranque más lentos comparado con frameworks minimalistas como Go o Rust.
- **Complejidad Oculta:** La "magia" de la auto-configuración puede dificultar la depuración si no se entiende bien qué está sucediendo por debajo.
- **Peso del Artefacto:** Los archivos generados (JAR) suelen ser pesados debido a la inclusión de todas las dependencias y el servidor embebido.

**Casos de Uso**

- **Cuando usarlo:** En **sistemas financieros, plataformas transaccionales**, microservicios complejos, y cualquier aplicación empresarial que requiera alta seguridad, escalabilidad y una estructura de capas bien definida (Controlador-Servicio-Repositorio).
- **Cuando NO usarlo:** En micro-herramientas extremadamente ligeras, funciones efímeras (Serverless) donde el tiempo de arranque es crítico, o aplicaciones con restricciones muy severas de memoria RAM.

**Ejemplos Reales**

1. **Sector Bancario:** La gran mayoría de los bancos internacionales utilizan Spring Boot para gestionar sus núcleos transaccionales debido a su robustez.

2. **Netflix:** Es uno de los mayores exponentes del uso de Spring Cloud (basado en Spring Boot) para gestionar su arquitectura de microservicios a escala global.
3. **LinkedIn:** Utiliza tecnologías del ecosistema Java/Spring para soportar gran parte de su infraestructura de servicios backend.
4. **Plataformas de E-commerce:** Muchas de las plataformas más grandes del mundo confían en Spring para gestionar catálogos, pagos y flujos críticos de compra.

## 3.3 Base de datos: Cassandra

**Definición**

Cassandra es un sistema de gestión de bases de datos NoSQL de tipo Wide-Column (columna ancha), diseñado originalmente por Facebook para manejar volúmenes masivos de datos distribuidos en muchos servidores básicos sin un punto único de fallo. A diferencia de las bases de datos relacionales tradicionales, Cassandra está optimizada para ofrecer una alta disponibilidad y escalabilidad horizontal masiva, permitiendo el almacenamiento de datos semiestructurados con un esquema flexible.

**Características**

- **Arquitectura Distribuida:** No utiliza un modelo maestro-esclavo; todos los nodos en un clúster tienen el mismo rol, lo que garantiza que no haya un punto único de falla.
- **Escalabilidad Horizontal:** Permite añadir nuevos nodos al clúster de forma sencilla para aumentar la capacidad de almacenamiento y procesamiento sin interrumpir el servicio.
- **Modelo de Datos Wide-Column:** Los datos se organizan en filas con un número variable de columnas, lo que permite una gran flexibilidad en comparación con las tablas rígidas de SQL.
- **Consistencia Ajustable:** Permite al desarrollador decidir el nivel de consistencia necesario para cada operación (por ejemplo, priorizar la velocidad de escritura sobre la consistencia inmediata).
- **Optimización de Escritura:** Está específicamente diseñada para manejar escrituras masivas distribuidas, lo cual es fundamental en entornos de Big Data.

**Ventajas y Desventajas**

**Ventajas:**

- **Alta Disponibilidad:** Gracias a su naturaleza distribuida, el sistema puede seguir funcionando incluso si varios nodos fallan simultáneamente.
- **Rendimiento en Escritura:** Es una de las bases de datos más rápidas del mercado para operaciones de inserción de datos a gran escala.
- **Tolerancia a Fallos:** Replica los datos automáticamente en diferentes nodos y, opcionalmente, en diferentes centros de datos.

**Desventajas:**

- **Complejidad en Consultas:** No soporta *joins* (uniones) ni subconsultas complejas de forma nativa, lo que obliga a diseñar el modelo de datos basado específicamente en las consultas que se van a realizar.
- **Consistencia Eventual:** En configuraciones de alto rendimiento, los datos pueden no estar sincronizados instantáneamente en todos los nodos.
- **Curva de Aprendizaje:** Requiere un cambio de mentalidad para desarrolladores acostumbrados al modelo relacional (SQL).

**Casos de Uso**

- **Cuando usarlo:** En sistemas que generan grandes volúmenes de datos en tiempo **real**, como registros de eventos (logs), telemetría de dispositivos **IoT**, aplicaciones de mensajería masiva y análisis de datos de redes sociales.
- **Cuando NO usarlo:** En aplicaciones con transacciones financieras complejas que requieren consistencia ácida (ACID) estricta, o en proyectos pequeños donde una base de datos relacional simple (como PostgreSQL) sería más que suficiente.

**Ejemplos Reales**

1. **Netflix:** Utiliza Cassandra para gestionar el historial de visualización y las preferencias de millones de usuarios simultáneamente en múltiples regiones geográficas.
2. **Apple:** Emplea miles de nodos de Cassandra para dar soporte a servicios masivos como iCloud y la App Store.
3. **Uber:** Confía en este motor para procesar y almacenar los datos de ubicación y estados de viajes en tiempo real de forma distribuida.
4. **Spotify:** Lo utiliza para gestionar metadatos de canciones y listas de reproducción que requieren actualizaciones constantes y acceso rápido.

## 3.4 Protocolo de comunicación: WebSocket

### Definición

WebSocket es un protocolo de comunicación que proporciona un canal de comunicación full-duplex (bidireccional y simultáneo) a través de una única conexión TCP de larga duración. A diferencia del modelo tradicional de solicitud-respuesta (como REST), WebSocket permite un flujo constante de datos donde tanto el cliente como el servidor pueden enviar información en cualquier momento sin necesidad de una petición previa. No es un protocolo sin estado; por el contrario, mantiene una conexión persistente que permanece abierta hasta que una de las partes decide cerrarla.

**Características Principales**

- **Conexión Persistente:** Establece un túnel de comunicación único que evita la sobrecarga de abrir y cerrar múltiples conexiones HTTP.
- **Comunicación Bidireccional:** Facilita que el servidor envíe actualizaciones proactivas ("push") al cliente sin esperar a que este las solicite.

- **Bajo "Overhead":** Tras el saludo inicial (*handshake*), los mensajes tienen encabezados mínimos, lo que reduce significativamente el consumo de ancho de banda en comparación con HTTP tradicional.
- **Compatibilidad con Protocolos de Mensajería:** Se integra frecuentemente con protocolos como **STOMP**, que es un protocolo de texto simple ideal para clientes ligeros y WebSockets.

**Historia y Evolución**

La necesidad de una comunicación bidireccional eficiente surgió ante las limitaciones del protocolo HTTP tradicional, diseñado originalmente bajo un modelo estrictamente síncrono de solicitud-respuesta. Antes de la existencia de WebSocket, los desarrolladores utilizaban técnicas ineficientes como el *Polling* (consultas constantes) o el *Long Polling*, que generaban una sobrecarga masiva en los servidores debido a la apertura y cierre constante de cabeceras HTTP.

La evolución cronológica del protocolo se resume en estos hitos clave:

- **Orígenes (2008-2009):** El concepto fue propuesto inicialmente como "TCPConnection" en las especificaciones de HTML5. Fue impulsado por ingenieros como Ian Hickson para permitir que las aplicaciones web tuvieran la misma capacidad de red que las aplicaciones de escritorio.
- **Estandarización (2011):** Tras varios borradores, el protocolo fue estandarizado oficialmente por la IETF como el RFC 6455. Esto permitió que los navegadores modernos implementaran una API nativa para manejar estas conexiones.
- **Protocolos sobre WebSockets:** Debido a que WebSocket es un protocolo de transporte de "bajo nivel" (solo envía mensajes), surgieron sub-protocolos para dar estructura a los datos. Un ejemplo fundamental es STOMP (Simple Text Oriented Messaging Protocol), que es ideal para clientes ligeros y facilita la integración con brokers de mensajería en el backend.
- **Evolución Moderna y Competencia:** Aunque WebSocket sigue siendo el estándar para conexiones full-duplex, han surgido alternativas modernas como HTTP/3 (basado en QUIC) para redes inestables y WebTransport, que se perfila como una evolución que ofrece menor latencia en ciertos escenarios de alto rendimiento.

**Ventajas y Desventajas**

**Ventajas:**

- **Latencia Mínima:** Al no tener que negociar la conexión en cada mensaje, la entrega de datos es prácticamente instantánea, ideal para streaming de datos.
- **Eficiencia de Recursos:** Consume menos recursos de red para aplicaciones que envían actualizaciones frecuentes.
- **Experiencia de Usuario:** Permite crear interfaces altamente reactivas que reflejan cambios en el servidor en tiempo real.

**Desventajas:**

- **Complejidad en el Servidor:** Mantener miles de conexiones abiertas requiere una gestión de memoria y concurrencia más sofisticada en el backend (Spring Boot).
- **Escalabilidad:** Los balanceadores de carga deben configurarse específicamente para manejar sesiones persistentes (*sticky sessions*).
- **Seguridad:** Al ser una conexión de larga duración, es vulnerable a ciertos tipos de ataques si no se implementan mecanismos de autenticación robustos como JWT.

**Casos de Uso**

- **Cuando usarlo:** En aplicaciones que requieren **comunicación en tiempo real**, como chats, tableros de control financiero, notificaciones instantáneas, juegos multijugador o herramientas de edición colaborativa.
- **Cuando NO usarlo:** Para aplicaciones de consulta simple de documentos estáticos o APIs donde el cliente solo necesita datos ocasionalmente y no requiere actualizaciones en tiempo real.

**Ejemplos Reales**

1. **Slack / WhatsApp Web:** Utilizan WebSockets para asegurar que los mensajes lleguen al destinatario en el momento exacto en que son enviados.
2. **Binance / Plataformas de Trading:** Emplean este protocolo para actualizar los precios de criptomonedas y acciones en milisegundos sin que el usuario refresque la página.
3. **Trello / Google Docs:** Permiten ver los movimientos y ediciones de otros colaboradores en el mismo instante en que ocurren.

## 3.5 Relación entre tecnologías

En este proyecto, las tecnologías seleccionadas no operan de forma aislada, sino que se integran bajo un modelo cliente-servidor para materializar los patrones de presentación MVC y MVVM. A continuación, se detalla cómo interactúa cada componente del stack:

**Cómo Angular consume Spring Boot (Integración vía WebSocket)**

La comunicación entre el frontend y el backend se aparta del modelo tradicional síncrono. Angular, actuando como el cliente, establece una conexión persistente y bidireccional con Spring Boot mediante el protocolo WebSocket.

- **Flujo de datos:** Angular utiliza la librería RxJS para suscribirse a flujos de datos asíncronos provenientes del servidor.
- **Reactividad:** Cuando ocurre un evento en el backend, Spring Boot envía un mensaje a través del canal de WebSocket; Angular recibe este mensaje y, gracias al mecanismo de data binding, actualiza la interfaz de usuario de forma instantánea sin necesidad de recargar la página.

**Cómo Spring Boot usa Cassandra (Persistencia)**

El backend implementado en Spring Boot actúa como la lógica de control y orquestación del sistema.

- **Acceso a datos:** Spring Boot utiliza el patrón Repository (específicamente a través de Spring Data Cassandra) para interactuar con la base de datos. Esto permite desacoplar la lógica de negocio de los detalles técnicos de la base de datos NoSQL.
- **Eficiencia:** Cassandra es utilizada por Spring Boot para gestionar escrituras masivas distribuidas, asegurando que el sistema pueda escalar horizontalmente mientras mantiene una alta disponibilidad de los datos de las tres entidades de negocio requeridas.

**Cómo encaja todo con el estilo MVC / MVVM**

El stack tecnológico se alinea con estos patrones de la siguiente manera:

1. **MVVM en el Frontend (Angular):** La Vista es la plantilla HTML; el ViewModel es el componente de TypeScript que gestiona el estado y la lógica de presentación; y el Modelo son los objetos de datos recibidos a través del WebSocket.
2. **MVC en el Sistema Global:**
    a. **Vista:** El frontend completo desarrollado en Angular.
    b. **Controlador:** Las clases @Controller o manejadores de mensajes en Spring Boot que reciben las peticiones del usuario y coordinan las acciones.
    c. **Modelo:** La lógica de negocio en los servicios de Java y la persistencia real en los nodos de Cassandra.

## 3.6 Popularidad del stack

El stack tecnológico compuesto por Angular, Spring Boot, Cassandra y WebSocket representa una de las combinaciones más robustas y vigentes en el desarrollo de software de grado empresarial. Esta selección no es casual; responde a una alta demanda en el mercado laboral y a la madurez de sus ecosistemas.

**Adopción en el Sector Empresarial**

La combinación de Angular y Spring Boot es considerada un estándar de facto en sectores críticos como el bancario, de seguros y salud.

- **Spring Boot (Java):** Es calificado como el "rey" del sector empresarial debido a su estabilidad a largo plazo y su capacidad para manejar lógica de negocio altamente compleja.
- **Angular:** Al ser un framework integral diseñado por Google para aplicaciones "Enterprise", garantiza soporte y disponibilidad de talento humano calificado, siendo uno de los "tres grandes" pilares del desarrollo frontend actual.

**Sinergia y Coherencia Técnica**

La relación entre estas tecnologías es muy común en arquitecturas que requieren manejar Big Data en tiempo real.

- **Uso de Cassandra:** Su popularidad ha crecido exponencialmente en sistemas de escritura masiva distribuida, siendo el motor preferido por gigantes como Netflix y Apple para garantizar que no existan puntos únicos de falla.
- **Integración vía WebSocket:** La adopción de WebSockets sobre el modelo tradicional REST responde a la tendencia actual de aplicaciones interactivas. Esta decisión técnica es fundamental para sistemas que requieren reflejar cambios de datos masivos (provenientes de Cassandra) en interfaces reactivas (Angular) de forma instantánea.

**Proyección en el Mercado Laboral**

Según indicadores como el StackOverflow Developer Survey y el TIOBE Index, este stack goza de una salud excepcional:

- **Demanda y Salarios:** Existe una brecha constante entre la alta demanda de desarrolladores Senior en Spring/Angular y la oferta disponible, lo que se traduce en niveles salariales competitivos y una proyección de crecimiento sostenido para el periodo 2026-1.
- **Ecosistema de Soporte:** Al ser tecnologías con comunidades masivas, la resolución de problemas y la disponibilidad de librerías complementarias es prácticamente ilimitada, reduciendo los riesgos de "Vendor Lock-in" u obsolescencia prematura.

**Casos de Éxito Combinados**

Empresas de Fintech e IoT utilizan frecuentemente esta combinación para construir tableros de control (dashboards) que procesan millones de eventos por segundo, donde Cassandra almacena el histórico, Spring Boot orquesta la seguridad y lógica, y Angular presenta los datos mediante WebSockets para una experiencia de usuario fluida.

# 4. Análisis Arquitectónico

## 4.1 Matriz: Atributos de calidad vs estilo

A continuación, se presenta el análisis de los atributos de calidad definidos por la norma ISO/IEC 25000:2014, evaluando cómo los patrones de presentación **MVC, MVP y MVVM** soportan o limitan cada uno de estos atributos dentro de una arquitectura cliente-servidor moderna.

**Adecuación funcional:** La adecuación funcional se refiere a la capacidad del sistema para proporcionar funciones que satisfagan las necesidades explícitas e implícitas de los usuarios, considerando la completitud, corrección y pertinencia de dichas funciones. Los patrones MVC, MVP y MVVM favorecen este atributo al permitir una clara separación entre la lógica de negocio y la interfaz, facilitando la correcta implementación de funcionalidades. Sin embargo, si existe una mala distribución de responsabilidades (por ejemplo, lógica en la vista), puede afectar la precisión funcional del sistema.

**Eficiencia de desempeño:** Este atributo evalúa el rendimiento del sistema en términos de tiempo de respuesta, uso de recursos y capacidad de procesamiento. En arquitecturas basadas en MVC/MVVM, el desempeño puede verse beneficiado por la separación de capas, permitiendo optimizar cada componente de manera independiente. No obstante, la introducción de múltiples capas y mecanismos como data binding en MVVM puede generar sobrecarga adicional si no se gestiona adecuadamente, afectando el rendimiento en aplicaciones de alta concurrencia.

**Compatibilidad:** La compatibilidad se refiere a la capacidad del sistema para interactuar con otros sistemas y coexistir en entornos compartidos. Los patrones de presentación facilitan este atributo al promover el uso de interfaces claras y desacopladas, especialmente en arquitecturas cliente-servidor donde el frontend y backend se comunican mediante APIs. Sin embargo, una implementación inadecuada puede generar dependencias implícitas que dificulten la interoperabilidad.

**Usabilidad:** La usabilidad evalúa qué tan fácil es para el usuario interactuar con el sistema, incluyendo aspectos como aprendizaje, eficiencia de uso y accesibilidad. MVVM, en particular, fortalece este atributo mediante el uso de data binding y componentes reactivos, permitiendo interfaces dinámicas y responsivas. MVC también contribuye al permitir múltiples vistas para un mismo modelo. No obstante, una mala implementación puede generar interfaces confusas o poco intuitivas.

**Fiabilidad:** La fiabilidad mide la capacidad del sistema para operar de manera continua y sin fallos, incluyendo tolerancia a errores y recuperación. La separación de responsabilidades en estos patrones contribuye a aislar errores y mejorar la estabilidad del sistema. Sin embargo, si existen componentes altamente acoplados o controladores sobrecargados, la propagación de fallos puede afectar múltiples partes del sistema.

**Seguridad:** Este atributo se enfoca en la protección de la información, garantizando confidencialidad, integridad y autenticidad. Los patrones MVC/MVVM no abordan directamente la seguridad, pero facilitan su implementación al permitir centralizar la lógica de control (por ejemplo, en controladores o servicios backend). Una mala implementación, como exponer lógica sensible en el frontend, puede comprometer la seguridad del sistema.

**Mantenibilidad:** La mantenibilidad es uno de los atributos más favorecidos por estos patrones, ya que promueven modularidad, bajo acoplamiento y alta cohesión. Esto facilita la modificación, análisis y pruebas del sistema. Sin embargo, si el patrón se implementa incorrectamente (por ejemplo, con God Objects o mala separación), puede generar el efecto contrario, dificultando el mantenimiento.

**Portabilidad:** La portabilidad se refiere a la capacidad del sistema para adaptarse a diferentes entornos o plataformas. Los patrones de presentación contribuyen indirectamente a este atributo al desacoplar la lógica de negocio de la interfaz, permitiendo cambiar tecnologías frontend o backend con menor impacto. No obstante, el uso de frameworks específicos puede limitar esta portabilidad si no se diseñan correctamente las abstracciones.

| Atributo de Calidad | Descripción | Cómo el estilo lo soporta | Cómo lo limita | Impacto |
|---|---|---|---|---|
| Adecuación funcional | Cumplimiento correcto de funciones | Separación MVC permite implementar lógica clara y organizada | Mala distribución de responsabilidades afecta precisión funcional | Alto |
| Eficiencia de desempeño | Uso eficiente de recursos y tiempos | Permite optimizar backend y frontend por separado | Capas adicionales y data binding pueden generar sobrecarga | Medio |
| Compatibilidad | Interacción con otros sistemas | Uso de APIs REST y desacoplamiento facilita interoperabilidad | Dependencias mal definidas pueden afectar integración | Alto |
| Usabilidad | Facilidad de uso del sistema | MVVM permite interfaces dinámicas y reactivas | Mala implementación genera interfaces confusas | Alto |
| Fiabilidad | Capacidad de operar sin fallos | Separación de componentes permite aislar errores | Acoplamiento o malas prácticas propagan fallos | Medio |
| Seguridad | Protección de datos | Centralización de lógica en backend facilita control de acceso | Exposición indebida en frontend compromete seguridad | Medio |
| Mantenibilidad | Facilidad de modificar el sistema | Alta modularidad y bajo acoplamiento | Mala implementación genera código difícil de mantener | Alto |
| Portabilidad | Adaptación a diferentes entornos | Separación permite cambiar tecnologías con menor impacto | Dependencia de frameworks limita migración | Medio |

## 4.2 Matriz: Principios vs estilo

Los patrones de presentación MVC, MVP y MVVM no solo organizan la estructura de la aplicación, sino que también facilitan la aplicación de principios fundamentales de diseño de software. A continuación, se analiza cómo estos estilos arquitectónicos permiten cumplir con principios clave como SOLID, DRY, KISS, YAGNI, PoLA, Ley de Demeter, STUPID y Composición sobre Herencia.

**SOLID**

El estilo MVC/MVVM facilita el cumplimiento de los principios SOLID, especialmente el principio de responsabilidad única (SRP), al separar claramente las responsabilidades entre modelo, vista y controlador o ViewModel. Asimismo, promueve el principio de inversión de dependencias (DIP) mediante el uso de interfaces y capas intermedias (como servicios o repositorios). Esta estructura modular permite que cada componente tenga una función específica, facilitando la extensibilidad y el mantenimiento del sistema sin afectar otras partes.

**DRY (Don't Repeat Yourself)**

Estos patrones promueven la reutilización de lógica mediante la centralización del comportamiento en componentes específicos como servicios, modelos o ViewModels. En lugar de duplicar lógica en múltiples vistas o controladores, se reutilizan estructuras comunes, reduciendo la redundancia y el riesgo de inconsistencias. Esto es especialmente evidente en arquitecturas con Angular y Spring Boot, donde los servicios encapsulan lógica reutilizable.

**KISS (Keep It Simple, Stupid)**

MVC y MVVM permiten mantener la simplicidad del sistema al dividirlo en componentes con responsabilidades claras, evitando estructuras monolíticas complejas. Cada parte del sistema tiene un propósito definido, lo que facilita su comprensión y desarrollo. Sin embargo, esta simplicidad depende de una correcta implementación, ya que una mala aplicación del patrón puede generar complejidad innecesaria.

**YAGNI (You Aren't Gonna Need It)**

El uso de estos patrones permite implementar únicamente las funcionalidades necesarias en cada capa del sistema, evitando la sobrecarga de lógica innecesaria. Al definir claramente las responsabilidades, se reduce la tentación de agregar funcionalidades futuras no requeridas, ya que cada componente se enfoca en cumplir un propósito específico dentro del flujo actual del sistema.

**PoLA (Principle of Least Astonishment)**

Estos patrones contribuyen a que el sistema sea predecible y fácil de entender, ya que siguen estructuras ampliamente conocidas y utilizadas en la industria. Los desarrolladores pueden anticipar dónde encontrar la lógica de negocio, la interfaz o el control de flujo, reduciendo la sorpresa o confusión al trabajar en el sistema.

**Ley de Demeter (Principle of Least Knowledge)**

La estructura de estos patrones favorece el bajo acoplamiento entre componentes, ya que cada capa interactúa únicamente con sus dependencias directas. Por ejemplo, los controladores interactúan con servicios y no directamente con la base de datos, lo que reduce el conocimiento innecesario entre componentes y mejora la mantenibilidad del sistema.

**STUPID (Anti-principios a evitar)**

El uso adecuado de MVC/MVVM ayuda a evitar los anti-patrones definidos en STUPID (como acoplamiento fuerte, duplicación o complejidad innecesaria). Al estructurar el sistema en capas y responsabilidades claras, se reduce la probabilidad de caer en malas prácticas como el uso excesivo de lógica en un solo componente o la duplicación de código.

**Composición sobre Herencia**

Estos patrones favorecen el uso de composición al estructurar la aplicación en componentes independientes que colaboran entre sí, en lugar de depender de jerarquías rígidas de herencia. Por ejemplo, en Angular, los componentes y servicios se combinan para construir funcionalidades complejas, lo que permite mayor flexibilidad y reutilización del código.

| Principio | Descripción | Cómo el estilo lo cumple | Aplicación en el contexto | Impacto |
|---|---|---|---|---|
| SOLID | Principios de diseño orientado a objetos | Separación en capas (modelo, vista, controlador) promueve SRP y DIP | Uso de controllers, services y repositories desacoplados | Alto |
| DRY | Evitar duplicación de código | Centralización de lógica en servicios y modelos reutilizables | Servicios en backend y frontend reutilizables | Medio |
| KISS | Mantener simplicidad | División clara de responsabilidades simplifica la comprensión del sistema | Estructura clara en Angular y Spring | Medio |
| YAGNI | No implementar lo innecesario | Cada componente implementa solo su responsabilidad específica | Evita lógica innecesaria en capas | Medio |
| PoLA | Comportamiento predecible | Estructura conocida y estándar facilita comprensión | Organización estándar en proyectos empresariales | Alto |
| Ley de Demeter | Bajo acoplamiento | Componentes interactúan solo con dependencias directas | Controladores → servicios → repositorios | Medio |

| | | | | |
|---|---|---|---|---|
| STUPID | Evitar malas prácticas | Estructura en capas reduce acoplamiento y duplicación | Prevención de God Objects y código desordenado | Medio |
| Composición sobre herencia | Preferir composición | Uso de componentes y servicios desacoplados | Angular y Spring usan inyección de dependencias | Alto |

## 4.3 Matriz: Tácticas vs estilo y stack

El análisis de tácticas arquitectónicas permite identificar las decisiones clave (ADR - Architecture Decision Records) que garantizan el cumplimiento de atributos de calidad dentro del sistema propuesto. En este caso, el sistema implementado corresponde a una aplicación cliente-servidor en tiempo real basada en WebSockets, donde el backend (Spring Boot) gestiona la lógica del juego y el frontend (Angular) actúa como vista reactiva.

Las decisiones arquitectónicas se centran en cómo el estilo de presentación (MVC/MVVM) y el stack tecnológico permiten resolver problemáticas relacionadas con rendimiento, escalabilidad, mantenibilidad, usabilidad y fiabilidad. Cada táctica representa una solución concreta a un problema arquitectónico, justificando el uso de tecnologías como WebSockets, Cassandra y la separación de responsabilidades en capas.

| Táctica | Atributo de calidad | Decisión (ADR) | Relación con el estilo (MVC/MVVM) | Implementación en el stack |
|---|---|---|---|---|
| Comunicación en tiempo real | Eficiencia / Usabilidad | Uso de WebSockets en lugar de HTTP tradicional | Separa la vista (Angular) del modelo actualizado en backend | WebSocketConfig en Spring Boot + cliente Angular |
| Separación de responsabilidades | Mantenibilidad | Uso de arquitectura en capas (controladores, servicios, repositorios) | MVC/MVVM separa vista, lógica y datos claramente | Angular (componentes + servicios) y Spring (Controller + Service + Repository) |
| Manejo de estado del sistema | Fiabilidad | Implementación de patrón State para estados del juego | Encapsula comportamiento del modelo sin afectar la vista | Clases EstadoJugando, EstadoPausado, etc. en backend |
| Persistencia distribuida | Escalabilidad | Uso de Cassandra para almacenamiento distribuido | El modelo se desacopla de la persistencia | Spring Data Cassandra + PartidaRepositorio |

| | | | | |
|---|---|---|---|---|
| Gestión de eventos del usuario | Usabilidad | Captura de eventos en frontend y envío al backend | Vista desacoplada del procesamiento de lógica | Angular captura eventos → envía DTO por WebSocket |
| Serialización de datos (DTOs) | Compatibilidad | Uso de DTOs para comunicación entre cliente y servidor | Evita acoplamiento entre modelo y vista | EstadoTableroDTO, MensajeAccionDTO |
| Contenerización | Portabilidad | Uso de Docker y Docker Compose | Independiente del estilo, soporta despliegue consistente | docker-compose.yml con backend + Cassandra |
| Manejo de errores centralizado | Fiabilidad | Uso de controladores globales de excepciones | Controladores MVC manejan errores sin afectar vista | @ControllerAdvice en Spring Boot |
| Renderizado reactivo | Usabilidad | Actualización dinámica del frontend basado en eventos | MVVM permite binding reactivo entre datos y vista | Angular con RxJS y suscripciones |
| Desacoplamiento frontend-backend | Compatibilidad | Comunicación mediante API y WebSockets | Permite independencia entre capas | Angular ↔ Spring Boot |

A continuación, se presentan algunas decisiones clave justificadas en el contexto del sistema:

**ADR 1: Uso de WebSockets para comunicación en tiempo real**

Se decide implementar WebSockets en lugar de un modelo tradicional basado en HTTP request-response. Esta decisión responde a la necesidad de mantener una comunicación bidireccional y de baja latencia entre el cliente y el servidor, fundamental en aplicaciones en tiempo real como videojuegos. Esta táctica mejora significativamente la eficiencia y la experiencia del usuario, y se integra de manera natural con el patrón MVC/MVVM, donde el frontend actúa como vista reactiva que se actualiza continuamente con los cambios del modelo.

**ADR 2: Centralización de la lógica en el backend**

Se establece que toda la lógica del juego (movimientos, colisiones, estados) se gestione exclusivamente en el backend. Esto garantiza consistencia, seguridad y control del sistema. Desde el punto de vista del estilo arquitectónico, esta decisión refuerza el principio de separación de responsabilidades, donde el modelo reside en el backend y la vista en el frontend, evitando duplicación de lógica y mejorando la mantenibilidad.

**ADR 3: Uso de Cassandra como base de datos distribuida**

Se selecciona Cassandra como motor de persistencia debido a su capacidad para manejar grandes volúmenes de escritura y su alta disponibilidad. Esta decisión está alineada con la necesidad de registrar partidas y eventos de manera eficiente. Además, el uso de un repositorio desacoplado del modelo permite mantener la flexibilidad del sistema sin afectar la lógica del negocio.

**ADR 4: Implementación del patrón State para manejo del juego**

Se adopta el patrón State para gestionar los diferentes estados de la partida (jugando, pausado, terminado). Esta decisión mejora la claridad y mantenibilidad del sistema, permitiendo encapsular comportamientos específicos sin afectar otras partes del código. Esta táctica se integra directamente con el modelo dentro del patrón MVC.

**ADR 5: Uso de Docker para portabilidad y despliegue**

Se decide contenerizar el backend y la base de datos mediante Docker Compose, lo que permite replicar el entorno de ejecución de manera consistente en diferentes máquinas. Esta decisión mejora la portabilidad y reduce problemas de configuración, siendo independiente del estilo arquitectónico pero fundamental para la implementación del sistema.

## 4.4 Matriz: Mercado laboral

El análisis del mercado laboral para el stack tecnológico definido (Angular, Spring Boot y Cassandra) evidencia que se trata de una combinación altamente relevante dentro del desarrollo de software empresarial moderno. Según reportes recientes de encuestas como Stack Overflow Developer Survey, GitHub Octoverse y JetBrains State of Developer Ecosystem, tecnologías como Java, TypeScript y frameworks asociados continúan posicionándose entre las más demandadas a nivel global. En particular, Angular se mantiene como un framework sólido en entornos corporativos, mientras que Spring Boot domina el backend en sectores como banca, seguros y sistemas empresariales (Stack Overflow, 2024; JetBrains, 2024).

Desde la perspectiva salarial, los desarrolladores que manejan este stack suelen ubicarse en rangos competitivos dentro del mercado internacional. De acuerdo con datos de plataformas como Glassdoor, Hired y el LinkedIn Economic Graph, un desarrollador con experiencia en Angular y Spring Boot puede alcanzar salarios anuales que oscilan entre USD $70,000 y $130,000 en mercados como Estados Unidos, mientras que en Latinoamérica los rangos pueden variar entre USD $2,000 y $5,000 mensuales, dependiendo de la experiencia y el nivel de especialización. El conocimiento de bases de datos distribuidas como Cassandra incrementa el valor del perfil, especialmente en contextos de Big Data y sistemas altamente escalables (Hired, 2024; LinkedIn, 2024).

En términos de demanda, Angular continúa siendo altamente valorado en aplicaciones empresariales de gran escala, donde la estructura robusta y el uso de TypeScript son ventajas clave. Por su parte, Spring Boot es considerado uno de los frameworks más utilizados en backend empresarial, respaldado por el ecosistema Java. Cassandra, aunque más especializado,

tiene una alta demanda en sectores que requieren procesamiento de grandes volúmenes de datos, como fintech, telecomunicaciones y plataformas de streaming. Esta combinación de tecnologías posiciona el stack como una opción sólida y vigente en el mercado actual (GitHub, 2024; TIOBE, 2024).

Finalmente, las tendencias del mercado apuntan hacia arquitecturas distribuidas, sistemas cloud-native y aplicaciones altamente escalables, donde este stack encaja de manera natural. Angular sigue evolucionando con mejoras en rendimiento y experiencia de desarrollo, Spring Boot se adapta a entornos de microservicios y cloud, y Cassandra continúa consolidándose como una solución clave para sistemas de alta disponibilidad. Esto indica que el stack no solo es relevante actualmente, sino que tiene una proyección favorable a mediano y largo plazo en la industria del software (IEEE Spectrum, 2024; Gartner, 2024).

| Tecnología | Demanda en el mercado | Salarios estimados | Tendencias actuales | Proyección |
|---|---|---|---|---|
| **Angular** | Alta en entornos empresariales y aplicaciones SPA robustas | USD $65,000 – $120,000/año (global) / $2,000 – $4,000 mensual (LATAM) | Uso con TypeScript, integración con arquitecturas modernas | Estable y sostenida en sector enterprise |
| **Spring Boot** | Muy alta (uno de los frameworks backend más usados) | USD $80,000 – $130,000/año / $2,500 – $5,000 mensual | Dominante en microservicios y backend empresarial | Muy alta, especialmente en cloud y banca |
| **Cassandra** | Media-alta (especializada en Big Data y sistemas distribuidos) | USD $90,000 – $140,000/año / $3,000 – $6,000 mensual | Uso en sistemas distribuidos, alta escalabilidad | Creciente en Big Data y sistemas globales |
| **Stack combinado** | Alta (perfiles full-stack enterprise muy valorados) | USD $90,000 – $150,000/año | Integración frontend-backend + datos distribuidos | Muy favorable a futuro |

# 5. Diseño del Sistema (C4 Model + HLD)

## 5.1 Descripción del sistema

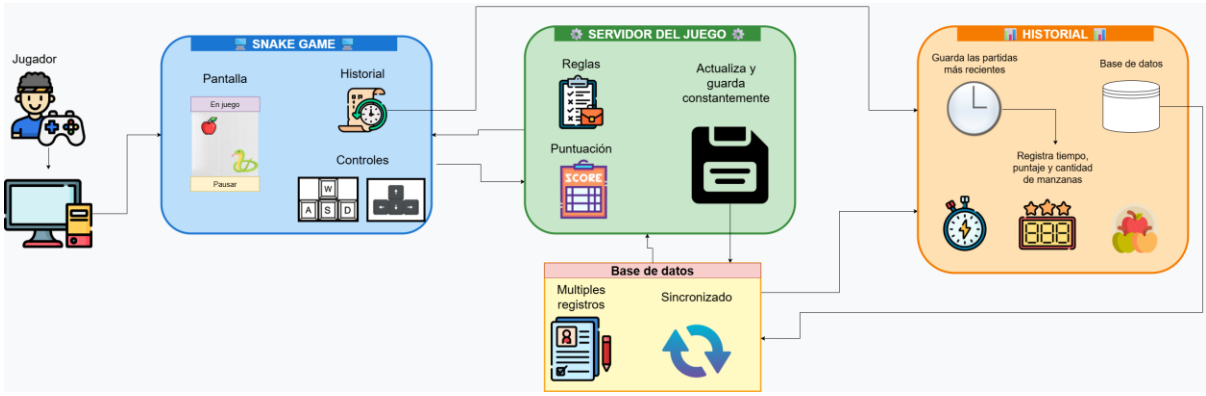
El sistema consiste en una versión moderna y reactiva del clásico juego "Snake" (culebrita), diseñada para demostrar la integración de estilos arquitectónicos distribuidos y patrones de presentación empresariales. La solución se basa en una arquitectura **cliente-servidor** que combina los patrones MVC (Model-View-Controller) en el backend y MVVM (Model-View-ViewModel) en el frontend para asegurar una separación clara de responsabilidades.

**Componentes Principales del Sistema:**

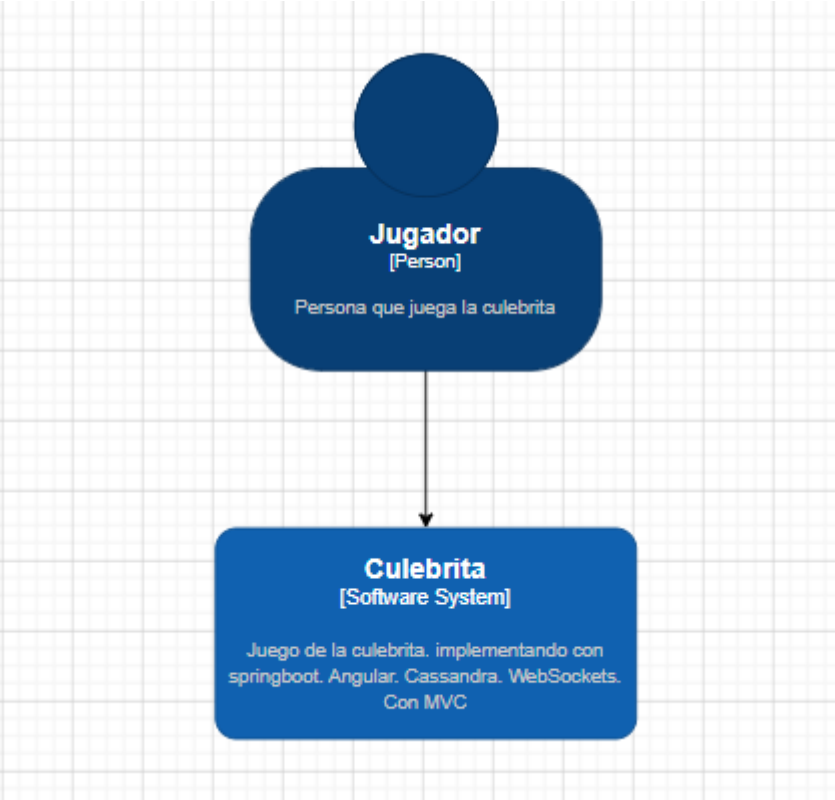
- **Frontend (Capa de Presentación):** Desarrollado en Angular (TypeScript), utiliza el patrón MVVM para gestionar la interfaz de usuario de forma reactiva. El *ViewModel* se encarga de la sincronización automática de los movimientos de la culebrita y el estado del tablero mediante data binding, proporcionando una experiencia de usuario fluida y dinámica.
- **Backend (Capa de Lógica de Negocio):** Implementado con Spring Boot (Java) bajo un esquema de MVC combinado con arquitectura de capas (Controller, Service, Repository, DTO y Modelos). Siguiendo las buenas prácticas, el controlador no posee lógica de negocio, limitándose a recibir solicitudes y delegar el procesamiento a la capa de servicios, donde reside la inteligencia del juego.
- **Lógica del Juego (Patrones de Diseño):** Para gestionar la complejidad de las mecánicas de juego, se implementó un patrón de diseño de Estado (State Pattern). Este patrón permite controlar las transiciones entre los diferentes estados del juego (como inicio, en juego, pausado o colisión) de manera organizada, evitando el uso excesivo de condicionales en una sola entidad y facilitando el mantenimiento del código.
- **Persistencia de Datos:** El sistema utiliza Cassandra, una base de datos NoSQL de tipo Wide-Column con arquitectura distribuida P2P (Peer-to-Peer). Aunque actualmente puede operar con una sola instancia, el diseño está orientado a la alta disponibilidad y escalabilidad horizontal, permitiendo que la información de puntajes y sesiones no esté centralizada en un único punto de falla.
- **Comunicación e Integración:** La interactividad en tiempo real se logra mediante el protocolo WebSockets, permitiendo una comunicación bidireccional y de baja latencia entre el frontend y el backend para el envío constante de coordenadas y estados del juego.
- **Despliegue y DevOps:** Todo el stack tecnológico está orquestado mediante Docker (Docker Compose). Esto garantiza que los contenedores de Angular, Spring Boot y Cassandra funcionen de manera aislada pero integrada, cumpliendo con el requisito obligatorio de portabilidad y facilidad de despliegue mediante un único comando.

Esta arquitectura no solo cumple con el flujo funcional de extremo a extremo (E2E) solicitado, sino que también justifica cada decisión técnica basándose en atributos de calidad como la mantenibilidad, escalabilidad y reactividad.

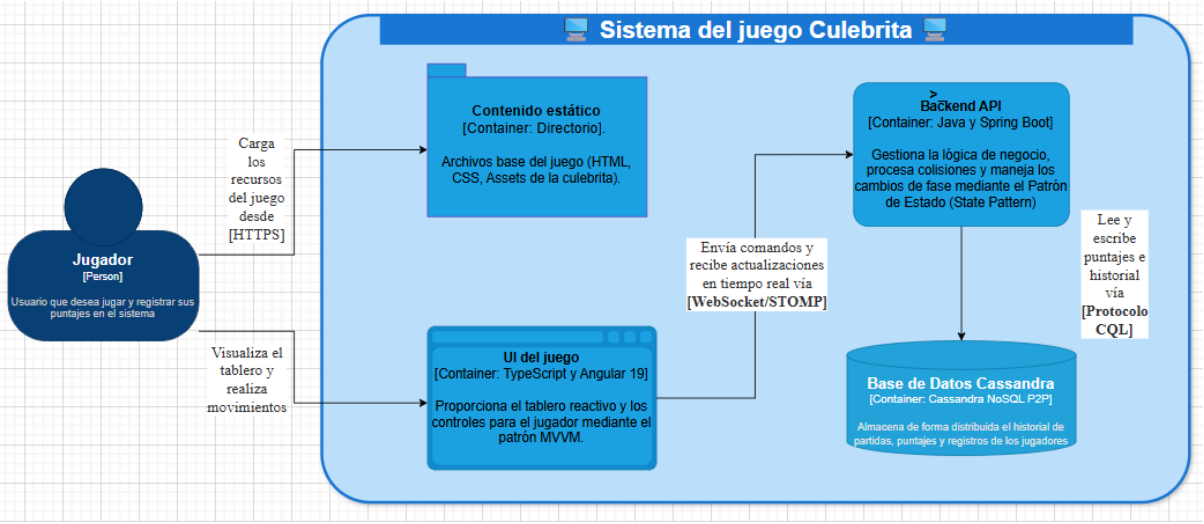
## 5.2 Diagrama de Alto Nivel (HLD)



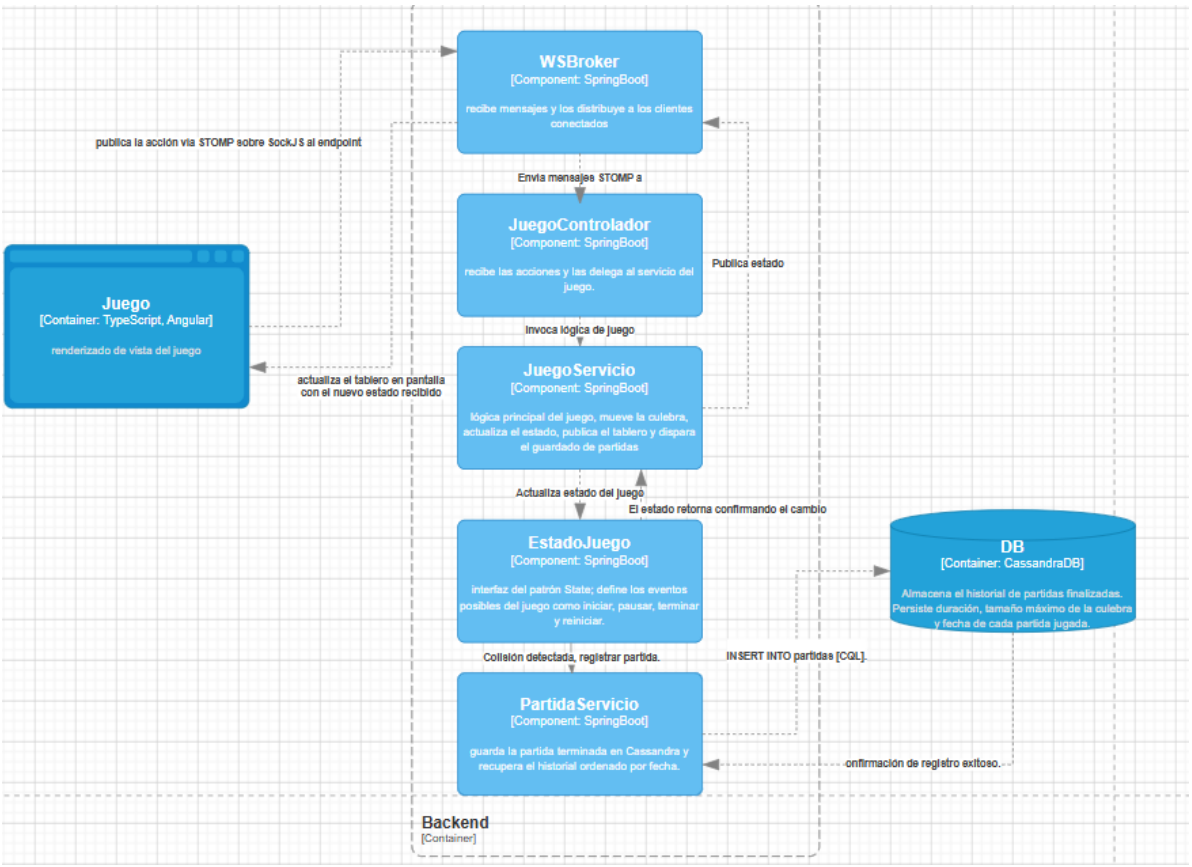

## 5.3 Diagrama de Contexto (Nivel 1)



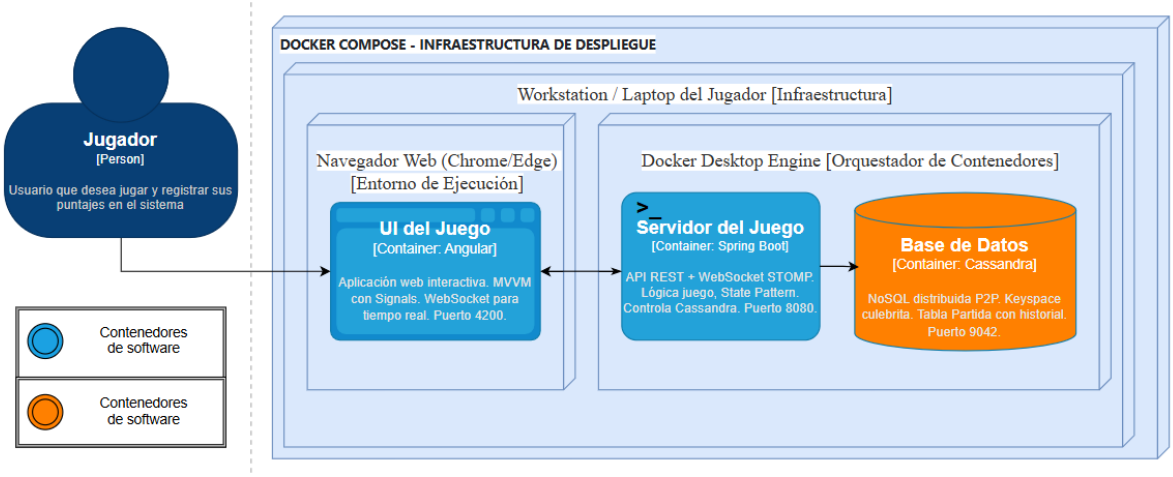
## 5.4 Diagrama de Contenedores (Nivel 2)




## 5.5 Diagrama dinámico



## 5.6 Diagrama de despliegue




# 6. Implementación

## 6.1 Descripción del proyecto

El sistema desarrollado corresponde a una implementación del clásico juego “Culebrita” (Snake) bajo un enfoque arquitectónico moderno, distribuido y desacoplado. A diferencia de su versión tradicional local, esta solución se construye como una aplicación cliente-servidor en tiempo real, donde el backend centraliza toda la lógica del juego y el frontend actúa como una interfaz de visualización e interacción. Esto permite no solo ejecutar partidas individuales, sino también escalar hacia escenarios con múltiples usuarios conectados simultáneamente y gestión persistente de partidas.

El núcleo funcional del sistema reside en el backend, el cual implementa una máquina de estados que gestiona el ciclo de vida de cada partida (esperando, jugando, pausado y terminado). A través de esta estructura, se controla de manera formal la evolución del juego, permitiendo transiciones seguras entre estados. La lógica incluye el cálculo de movimientos de la culebra, generación de comida, detección de colisiones y actualización de la puntuación. Cada cambio en el estado del juego es procesado en el servidor y transmitido al cliente en tiempo real.

El frontend, desarrollado con Angular, cumple el rol de vista reactiva, encargándose de representar gráficamente el estado del juego y capturar las acciones del usuario (como eventos de teclado). Estas acciones son enviadas al backend mediante WebSockets, lo que permite una comunicación bidireccional persistente y de baja latencia. De esta manera, el sistema evita el modelo tradicional request-response, logrando una experiencia fluida y continua sin recargas de página.

El sistema incorpora persistencia mediante una base de datos NoSQL (Cassandra), donde se almacenan los resultados de las partidas. Esta elección tecnológica permite soportar altas tasas

de escritura y garantizar disponibilidad, características clave en sistemas con potencial crecimiento. En conjunto, el proyecto no solo implementa un juego funcional, sino que demuestra la aplicación práctica de principios de arquitectura moderna, separación de responsabilidades y comunicación en tiempo real.

## 6.2 Arquitectura implementada

La arquitectura implementada sigue un enfoque basado en patrones de presentación (MVC/MVVM) integrados en una arquitectura cliente-servidor con comunicación en tiempo real mediante WebSockets, complementada con una organización por capas en el backend. Esta combinación permite una clara separación de responsabilidades entre la lógica de negocio, la interfaz de usuario y la persistencia, alineándose con buenas prácticas de diseño de software.

En el frontend (Angular), se adopta un enfoque cercano a MVVM, donde los componentes actúan como la vista, los servicios gestionan la lógica de comunicación y los modelos representan el estado de los datos. La integración con RxJS permite manejar flujos reactivos de información provenientes del backend, lo que resulta clave en un sistema en tiempo real. El frontend no contiene lógica de negocio, limitándose a renderizar el estado recibido y enviar acciones del usuario, lo que garantiza bajo acoplamiento.

En el backend (Spring Boot), se implementa una estructura tipo MVC por capas, organizada en controladores, servicios, modelos y repositorios.

- Los controladores (REST y WebSocket) actúan como punto de entrada.
- Los servicios encapsulan la lógica de negocio del juego.
- Los modelos representan las entidades del dominio (culebra, partida, comida, etc.).
- Los repositorios gestionan la persistencia en Cassandra mediante Spring Data.

Adicionalmente, se implementa el patrón State para modelar la máquina de estados del juego, lo que mejora la mantenibilidad y claridad del flujo lógico.

La comunicación entre frontend y backend se realiza mediante WebSockets, permitiendo un flujo bidireccional continuo. Este mecanismo es fundamental para garantizar baja latencia en la actualización del estado del juego. Cada acción del usuario se envía al servidor, donde se procesa y genera un nuevo estado del tablero, el cual es inmediatamente retransmitido al cliente.

En cuanto a la persistencia, Cassandra se integra como una base de datos distribuida orientada a columnas, ideal para escenarios con alta disponibilidad y escrituras intensivas. El acceso a datos se abstrae mediante repositorios, evitando acoplamiento directo con la base de datos y facilitando la evolución del sistema.

La arquitectura se despliega mediante contenedores Docker, orquestados con Docker Compose, lo que permite ejecutar de manera integrada el frontend, backend y base de datos en una misma red virtual. Esta estrategia garantiza portabilidad, reproducibilidad del entorno y alineación con prácticas modernas de DevOps. En conjunto, la arquitectura implementada

refleja un sistema escalable, desacoplado y preparado para entornos distribuidos y en tiempo real.

## 6.3 Flujo funcional

El flujo funcional del sistema se basa en una arquitectura cliente-servidor en tiempo real, donde la interacción entre frontend, backend y base de datos se realiza de forma continua mediante WebSockets y operaciones REST complementarias. Este flujo inicia cuando el cliente (Angular) carga la aplicación y establece una conexión persistente con el backend (Spring Boot) a través del endpoint WebSocket configurado.

- **Inicialización y conexión:**
  - El componente principal del frontend (tablero) se inicializa y, a través de juego.servicio.ts, establece una conexión WebSocket hacia el backend (por ejemplo, ws://localhost:8080/ws).
  - El backend, mediante la configuración definida en WebSocketConfig.java, acepta la conexión y asocia la sesión del cliente a una instancia de partida activa o nueva.
- **Inicio de partida:**
  - El frontend puede realizar una solicitud HTTP REST hacia PartidaControlador.java para crear o recuperar una partida.
  - El backend instancia el modelo Partida.java y su contexto de estado (EstadoJuego.java), inicializando la culebra, la comida y el tablero.
- **Interacción del usuario:**
  - El usuario interactúa mediante eventos de teclado capturados en tablero.component.ts.
  - Estas acciones son transformadas en objetos MensajeAccionDTO.java y enviadas al backend a través del canal WebSocket.
- **Procesamiento en backend:**
  - El backend recibe el mensaje en JuegoControlador.java o en el manejador WebSocket correspondiente.
  - Se delega la lógica al servicio JuegoServicio.java, el cual:
    - Actualiza la dirección de la culebra.
    - Calcula el siguiente estado del tablero.
    - Evalúa colisiones, consumo de comida y cambios de estado.
    - Gestiona la transición de estados mediante el patrón **State** (EstadoEsperando, EstadoJugando, etc.).
- **Persistencia de datos:**
  - Cuando ocurre un evento relevante (por ejemplo, finalización de partida), el PartidaServicio.java persiste los datos mediante PartidaRepositorio.java en Cassandra.
  - Cassandra almacena la información de manera distribuida en el keyspace definido, garantizando alta disponibilidad y tolerancia a fallos.

- **Actualización del cliente:**
    - Tras cada iteración del juego, el backend construye un EstadoTableroDTO.java con la información actualizada.
    - Este DTO se envía por WebSocket al frontend.
    - El cliente actualiza su modelo local (estado-tablero.modelo.ts) y renderiza visualmente el nuevo estado del juego.

Este flujo se ejecuta de manera continua en ciclos muy cortos, logrando una experiencia en tiempo real. La separación entre procesamiento (backend) y visualización (frontend) garantiza escalabilidad, mientras que el uso de WebSockets reduce la latencia y elimina la sobrecarga del modelo HTTP tradicional.

## 6.4 Manejo de errores y validaciones

El sistema implementa un conjunto de estrategias de manejo de errores y validaciones distribuidas entre frontend y backend, con el objetivo de garantizar consistencia, robustez y una experiencia de usuario controlada frente a fallos.

- **Validaciones en frontend (Angular):**
    - Se validan las entradas del usuario antes de ser enviadas al backend, evitando acciones inválidas como direcciones opuestas inmediatas (ej. ir de izquierda a derecha instantáneamente).
    - Los servicios (juego.servicio.ts, partida.servicio.ts) controlan el estado de la conexión WebSocket, manejando eventos como desconexión o reconexión.
    - Se implementa control básico de errores en la suscripción a observables (RxJS), permitiendo capturar fallos en la comunicación y evitar bloqueos en la interfaz.
- **Validaciones en backend (Spring Boot):**
    - Los controladores REST (PartidaControlador.java) validan la integridad de las solicitudes HTTP (por ejemplo, existencia de IDs de partida).
    - En el procesamiento WebSocket, el backend valida la estructura de los mensajes recibidos (MensajeAccionDTO) y descarta aquellos que no cumplen con el formato esperado.
    - La lógica de negocio en JuegoServicio.java incluye validaciones de reglas del juego, como colisiones, límites del tablero y estados válidos de transición.
- **Manejo de errores en lógica de negocio:**
    - Se controla el flujo mediante la máquina de estados, evitando transiciones inválidas (por ejemplo, jugar cuando la partida está terminada).
    - Se previenen inconsistencias mediante encapsulación de la lógica en servicios, reduciendo efectos secundarios no controlados.
- **Manejo de errores en persistencia (Cassandra):**
    - El acceso a datos mediante PartidaRepositorio.java se apoya en Spring Data, que gestiona excepciones relacionadas con la base de datos.

    - Se consideran escenarios de consistencia eventual, donde los datos pueden no reflejarse inmediatamente, pero se garantiza disponibilidad del sistema.
- **Manejo de errores de infraestructura (Docker y red):**
    - Docker Compose permite reiniciar servicios automáticamente en caso de fallo (según configuración).
    - La comunicación entre servicios se realiza por nombre de contenedor, reduciendo errores de configuración de red.
    - Se mitiga el problema "works on my machine" al estandarizar el entorno de ejecución.

## 6.5 Integración entre componentes

La integración entre componentes del sistema se basa en un modelo distribuido desacoplado, donde cada capa (frontend, backend y persistencia) interactúa mediante interfaces bien definidas y protocolos específicos. Esta integración garantiza coherencia funcional, comunicación eficiente en tiempo real y escalabilidad del sistema, manteniendo una clara separación de responsabilidades.

A nivel general, la arquitectura implementa tres tipos principales de integración:

- **Integración frontend ↔ backend (comunicación en tiempo real y síncrona)**:
    - Se implementa mediante WebSockets para la comunicación bidireccional en tiempo real, permitiendo el envío continuo de eventos del cliente al servidor y viceversa.
    - El frontend (Angular), a través de juego.servicio.ts, establece una conexión persistente con el backend configurado en WebSocketConfig.java.
    - Los mensajes enviados por el cliente (MensajeAccionDTO) representan acciones del usuario (direcciones del juego), mientras que las respuestas del backend (EstadoTableroDTO) contienen el estado actualizado del sistema.
    - Adicionalmente, se utilizan endpoints REST (en PartidaControlador.java) para operaciones no críticas en tiempo real, como la creación o consulta de partidas.
- **Integración backend ↔ lógica de negocio**:
    - El backend está estructurado bajo una arquitectura por capas, donde los controladores delegan las solicitudes a los servicios, desacoplando la lógica de transporte de la lógica de negocio.
    - JuegoControlador.java y los manejadores WebSocket reciben eventos y los delegan a JuegoServicio.java, que encapsula la lógica central del juego.
    - La implementación del patrón State (clases en el paquete estado) permite gestionar dinámicamente el comportamiento del sistema según el estado actual de la partida, asegurando una integración coherente entre lógica y flujo del sistema.
- **Integración backend ↔ persistencia (Cassandra)**:
    - Se realiza mediante el uso de Spring Data Cassandra, a través del repositorio PartidaRepositorio.java.

    - El servicio PartidaServicio.java actúa como intermediario entre la lógica de negocio y la base de datos, gestionando operaciones de escritura y consulta.
    - La base de datos Cassandra, configurada en CassandraKeyspaceConfig.java, permite almacenar información de partidas de manera distribuida y altamente disponible.
    - Esta integración está diseñada bajo el principio de bajo acoplamiento, permitiendo sustituir el motor de base de datos sin afectar la lógica del sistema.

A nivel de flujo integrado, la interacción completa entre componentes ocurre de la siguiente manera:

- El frontend captura eventos del usuario y los envía al backend mediante WebSockets.
- El backend procesa estos eventos en la capa de servicios, actualizando el modelo del dominio (culebra, tablero, estados).
- Si ocurre un evento relevante (como finalización de partida), se invoca la capa de persistencia para almacenar los datos en Cassandra.
- El backend genera un nuevo estado del sistema y lo envía nuevamente al frontend, cerrando el ciclo de comunicación.
- El frontend renderiza el nuevo estado, manteniendo sincronización en tiempo real con el servidor.

En términos de infraestructura, la integración entre componentes se soporta mediante Docker y Docker Compose, lo que permite que todos los servicios se ejecuten dentro de una misma red virtual:

- Cada componente (frontend, backend y Cassandra) se ejecuta en un contenedor independiente.
- La comunicación entre servicios se realiza mediante nombres de servicio definidos en docker-compose.yml (por ejemplo, backend:8080 o cassandra:9042), eliminando dependencias de configuración manual.
- Esta configuración asegura portabilidad del sistema, consistencia entre entornos y facilidad de despliegue.

# 7. Repositorio y DevOps

- Enlace del repositorio: https://github.com/juanealop/Culebrita.git

**Uso de Docker**

La implementación del sistema se apoya en Docker y Docker Compose como mecanismos de contenerización y orquestación de servicios, permitiendo garantizar portabilidad, consistencia entre entornos y facilidad de despliegue. Cada componente del sistema (frontend en Angular, backend en Spring Boot y base de datos Cassandra) se ejecuta en contenedores independientes

definidos mediante archivos Dockerfile específicos para frontend y backend, y un archivo docker-compose.yml que orquesta la ejecución conjunta.

Docker permite encapsular todas las dependencias necesarias para la ejecución del sistema, eliminando problemas de configuración entre distintos entornos de desarrollo. A través de Docker Compose, se define una red interna donde los servicios pueden comunicarse mediante nombres de host (por ejemplo, el backend accede a Cassandra mediante cassandra:9042), facilitando la integración entre componentes. Este enfoque asegura que el sistema pueda desplegarse de manera uniforme tanto en entornos locales como en servidores remotos, alineándose con buenas prácticas DevOps.

**README (Descripción, tecnologías y despliegue)**

El proyecto implementa un sistema del juego "Culebrita" (Snake) bajo una arquitectura cliente-servidor en tiempo real, donde la lógica del juego se ejecuta completamente en el backend y el frontend actúa como una interfaz de visualización y control. El sistema permite gestionar partidas, procesar acciones del usuario en tiempo real y almacenar el historial de resultados, utilizando una máquina de estados para controlar el ciclo de vida de cada partida.

En cuanto a las tecnologías utilizadas, el frontend está desarrollado con Angular, TypeScript y Tailwind CSS, lo que permite construir una interfaz reactiva y modular. El backend utiliza Spring Boot con Java, implementando controladores REST y comunicación mediante WebSockets para garantizar interacción en tiempo real. La persistencia se maneja con Cassandra, una base de datos NoSQL distribuida optimizada para alta disponibilidad y grandes volúmenes de escritura. Adicionalmente, Docker y Docker Compose se utilizan para la contenerización y despliegue del sistema.

Para el despliegue del sistema, se debe clonar el repositorio y ejecutar Docker Compose desde la raíz del proyecto correspondiente. Este proceso construye las imágenes necesarias y levanta los contenedores de frontend, backend y base de datos en una red compartida. Una vez en ejecución, el backend queda disponible en el puerto configurado (por ejemplo, 8080), y el frontend puede acceder a este para establecer la conexión WebSocket y comenzar la interacción del juego. Este flujo permite ejecutar el sistema completo sin necesidad de instalar manualmente las dependencias, asegurando reproducibilidad y facilidad de uso.

# 8. Lecciones Aprendidas

**Qué funcionó**

La adopción de una arquitectura basada en separación de responsabilidades (MVC en backend y MVVM en frontend) permitió estructurar el sistema de forma clara y escalable, facilitando el desarrollo colaborativo. La utilización de **WebSockets** fue una decisión clave, ya que permitió cumplir con los requerimientos de tiempo real del juego, logrando una comunicación eficiente y de baja latencia entre Angular y Spring Boot. Asimismo, el uso de **Cassandra**

resultó adecuado para manejar persistencia orientada a escritura, especialmente para el historial de partidas, y la contenerización con Docker garantizó portabilidad y consistencia en los entornos de ejecución.

**Qué no funcionó**

Uno de los principales retos fue la complejidad inicial en la integración de múltiples tecnologías, especialmente en la configuración de WebSockets y la comunicación bidireccional en tiempo real. También se evidenciaron dificultades en la correcta distribución de responsabilidades, principalmente al evitar sobrecargar servicios o controladores en el backend. Adicionalmente, Cassandra implicó un cambio de paradigma frente a bases de datos relacionales, lo que generó dificultades en el modelado inicial de datos y en la comprensión de la consistencia eventual.

**Qué mejorarían**

Se mejoraría la implementación incorporando mecanismos más robustos de manejo de errores y observabilidad, como logging estructurado y monitoreo en tiempo real. También sería recomendable fortalecer la seguridad del sistema, integrando autenticación (por ejemplo, JWT en WebSockets). A nivel arquitectónico, se podría optimizar la gestión del estado del frontend mediante herramientas especializadas (como NgRx) para mejorar la escalabilidad. Finalmente, se propondría una mejor automatización del despliegue (CI/CD) y pruebas más completas (unitarias e integración) para garantizar mayor calidad del software.

# 9. Conclusiones

- La implementación del sistema demostró que la combinación de patrones arquitectónicos como MVC y MVVM permite construir aplicaciones altamente desacopladas, facilitando la mantenibilidad, escalabilidad y evolución del software en entornos reales.
- El uso de tecnologías modernas como Angular, Spring Boot, Cassandra y WebSockets evidenció ser una elección adecuada para aplicaciones en tiempo real, logrando una integración eficiente entre frontend, backend y persistencia.
- La arquitectura cliente-servidor con comunicación bidireccional permitió separar completamente la lógica de negocio del frontend, garantizando que el backend sea el núcleo del sistema y permitiendo la reutilización con múltiples clientes.
- La contenerización mediante Docker no solo facilitó el despliegue del sistema, sino que también eliminó problemas de compatibilidad entre entornos, evidenciando su importancia en prácticas modernas de desarrollo.
- El desarrollo del proyecto permitió comprender que una buena arquitectura no depende únicamente de las tecnologías utilizadas, sino de cómo se aplican los principios de diseño, la correcta separación de responsabilidades y la toma de decisiones alineadas con los requisitos del sistema.

# 10. Referencias


- Buschmann, F., Meunier, R., Rohnert, H., Sommerlad, P., & Stal, M. (1996). Pattern-Oriented Software Architecture: A System of Patterns. Wiley. https://ribeaud.github.io/SWA/lectures/8/pattern_oriented_software_architecture.pdf
- Fowler, M. (2004). Patterns of Enterprise Application Architecture. Addison-Wesley. https://martinfowler.com/eaaCatalog/
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley. https://www.javier8a.com/itc/bd1/articulo.pdf
- Microsoft. (2012). MVVM Pattern. Microsoft Docs. https://learn.microsoft.com/en-us/archive/msdn-magazine/2012/february/mvvm-design-pattern
- Freeman, A., & Robson, E. (2014). Pro AngularJS. Apress. https://archive.org/details/isbn_9788132230960/page/n3/mode/2up
- Krasner, G. E., & Pope, S. T. (1988). A cookbook for using the model-view controller user interface paradigm in Smalltalk-80. Journal of Object-Oriented Programming. https://www.lri.fr/~mbl/ENS/FONDIHM/2013/papers/Krasner-JOOP88.pdf
- VLink. (2024). *Angular Development Success Stories: Real-world Examples*. https://vlinkinfo.com/blog/success-stories-with-angular-development-services/
- Dimension Zero. (2024). *MVC vs MVVM: Real-world case studies*. https://dev.to/dimension-zero/mvc-vs-mvvm-deep-dive-into-real-world-flow-patterns-part-5-4o28
- Gothinkster. (2024). *Angular RealWorld Example App*. https://github.com/gothinkster/angular-realworld-example-app
- Stack Overflow. (2024). *Developer Survey Results*. https://survey.stackoverflow.co/2024
- JetBrains. (2024). *State of Developer Ecosystem*. https://www.jetbrains.com/lp/devecosystem-2024/
- GitHub. (2024). *Octoverse Report*. https://octoverse.github.com
- TIOBE. (2024). *TIOBE Index*. https://www.tiobe.com/tiobe-index/
- IEEE Spectrum. (2024). *Top Programming Languages*. https://spectrum.ieee.org/top-programming-languages
- Gartner. (2024). *Technology Trends Report*. https://www.gartner.com/en/information-technology
- Hired. (2024). *State of Software Engineers*. https://hired.com/state-of-software-engineers
- LinkedIn. (2024). *Economic Graph*. https://economicgraph.linkedin.com

- Google. (2023). *Angular documentation*. Recuperado de https://angular.io/docs
- VMware. (2024). *Spring Boot Reference Documentation*. Recuperado de https://docs.spring.io/spring-boot/docs/current/reference/html/
- Apache Software Foundation. (2024). *Apache Cassandra Documentation*. Recuperado de https://cassandra.apache.org/doc/latest/
- Docker Inc.. (2024). *Docker Documentation*. Recuperado de https://docs.docker.com/